DATA STRUCTURES LAB 3

MUHAMMAD TAHA
22F-3277
BCS-3C

TASK 1

```
//Program to reverse a linked list
#include <iostream>
using namespace std;
class Node
{
public:
	int data;
	Node* next;
	Node(int value) :data(value), next(NULL) {}
};

class List
{
public:
	Node* head;
	List() :head(NULL) {}
	void create(int numOfNodes)
	{
		for (int i = 0; i < numOfNodes; i++)
		{
			int value;
			cout << "Enter value for node " << i + 1<<" :";
			cin >> value;

			Node* newNode = new Node (value);	//create new node

			if (head == NULL)
			{
				head = newNode;					//if the
list is empty make newNode the head
				newNode->next = NULL;			//next node of
head is null
			}
			else
			{
				//if the list is not empty make newNode at the end of
head
				Node* current = head;
				while (current->next != NULL)
				{
					current = current->next;
				}
				current->next = newNode;	//newNode is added at
the end of list
				newNode->next = NULL;			//next will be
Null unless newNode is added


			}
		}
	}

	void reverse()
	{
		Node* current = head;
		Node* prev = NULL, *nextNode = NULL;
		while (current != NULL)
		{
			nextNode = current->next; //the next of the current node is
made nextNode
```

```
                    current->next = prev;           //current nodes now points to
previous node
                    prev = current;                       //current is made
prevvious so that when we move forward we have the address of previous
                    current = nextNode;             //nextNode is made current to
move forward
             }
             head = prev;
      }

      void display()
      {
             Node* current = head;
             int i = 1;
             while (current != NULL)
             {
                    cout << "Value at node " << i++ << " is " << current->data <<
endl;
                    current = current->next;
             }
      }
};

int main()
{
      int num;
      List l;

      cout << "Enter number of nodes to create: ";
      cin >> num;
      l.create(num);
      l.display();

      cout << "To Reverse ";
      system("pause");

      cout << endl << endl;
      l.reverse();
      l.display();

      system("pause");
      return 0;

}
```

```
Enter number of nodes to create: 5
Enter value for node 1 :8
Enter value for node 2 :6
Enter value for node 3 :9
Enter value for node 4 :7
Enter value for node 5 :2
Value at node 1 is 8
Value at node 2 is 6
Value at node 3 is 9
Value at node 4 is 7
Value at node 5 is 2
To Reverse Press any key to continue . . .


Value at node 1 is 2
Value at node 2 is 7
Value at node 3 is 9
Value at node 4 is 6
Value at node 5 is 8
Press any key to continue . . .

C:\Users\muham\OneDrive\Desktop\CODING PRAC
```

Figure 1 Reversing Linked List

TASK 2

```
//Program to reverse a linked list
#include <iostream>
using namespace std;
class Node
{
public:
	int data;
	Node* next;
	Node(int value) :data(value), next(NULL) {}
};

class List
{
public:
	Node* head;
	List() :head(NULL) {}
	void create(int numOfNodes)
	{
		for (int i = 0; i < numOfNodes; i++)
		{
			int value;
			cout << "Enter value for node " << i + 1<<" :";
			cin >> value;

			Node* newNode = new Node (value);	//create new node

			if (head == NULL)
			{
				head = newNode;					//if the
list is empty make newNode the head
				newNode->next = NULL;			//next node of
head is null
			}
			else
			{
				//if the list is not empty make newNode at the end of
head
				Node* current = head;
				while (current->next != NULL)
				{
					current = current->next;
				}
				current->next = newNode;	//newNode is added at
the end of list
				newNode->next = NULL;			//next will be
Null unless newNode is added


			}
		}
	}

	bool binarySearch(int target)
	{
		Node* left = head;  //left points to start(head) of list
		Node* right = NULL; //right points to end of list

		//find size of list
		int size = 0;
		Node* temp = head;
```

```
		while (temp != NULL)
		{
			size++;
			temp = temp->next;
		}

		Node* current = head;
		//binary search
		while (left != right)
		{
			int mid = size / 2;	//finding the mid
			current = left;			//left is assigned to current


			for (int i = 0; i < mid; i++)	//reaching till mid of the
list
				current = current->next;

			if (current->data == target)		//if element is found
at return true
				return true;

			//if current data is less than target than is means that the
target is towards right so next is assigned to left
			else if (current->data < target)
				left = current->next;

			//if current data is greater than target than is means that
the target is towards left, so current is assigned to right to limit the end
			else //if (current->data > target)
				right = current;

			//calculating the size again
			size = size - mid;
		}

		//returns false if data not found
		return false;
	}

	//display
	void display()
	{
		Node* current = head;
		int i = 1;
		while (current != NULL)
		{
			cout << "Value at node " << i++ << " is " << current->data <<
endl;
			current = current->next;
		}
	}
};

int main()
{
	int num;
	List l;

	cout << "Enter number of nodes to create: ";
	cin >> num;
	l.create(num);
	l.display();
```

```
	cout << endl << endl;
	int target;

	cout << "Enter the element to search for: ";
	cin >> target;

	if (l.binarySearch(target))
		cout << target << " found in the list\n";
	else
		cout << target << " not found in the list\n";

	system("pause");
	return 0;
}
```

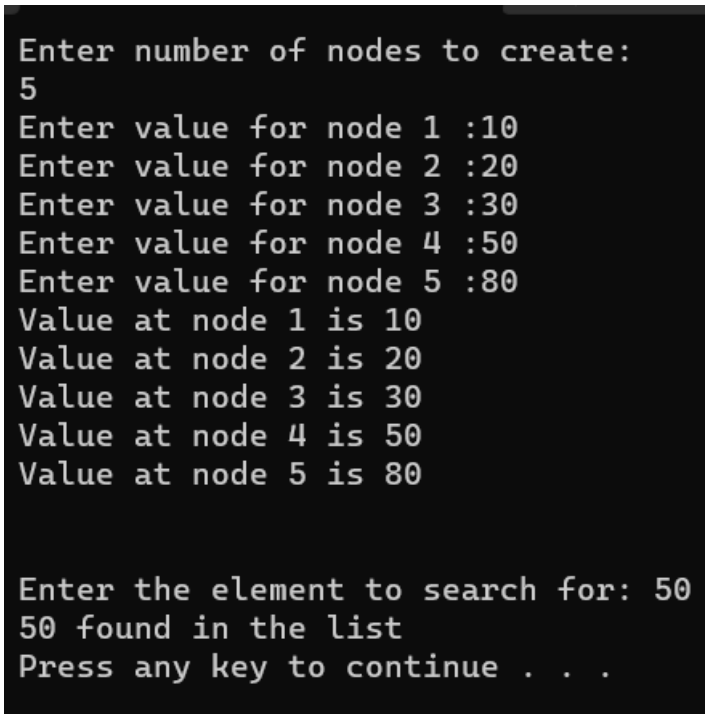


Figure 2 Element is found

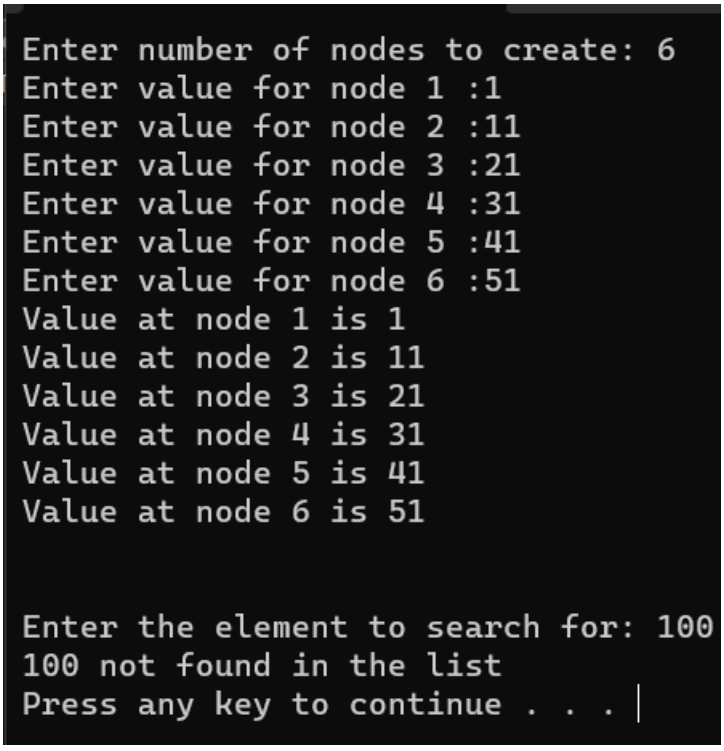


Figure 3 Element not Found

TASK 3

```
//Program to group all odd nodes together followed by the even nodes in linked
list
#include <iostream>
using namespace std;
//Node Class
class Node
{
public:
	int data;
	Node* next;
	Node(int value) :data(value), next(NULL) {}
};
class List
{
public:
	Node* head;
	List() :head(NULL) {}
	void create(int numOfNodes)
	{
		for (int i = 0; i < numOfNodes; i++)
		{
			int value;
			cout << "Enter value for node " << i + 1 << " : ";
			cin >> value;

			Node* newNode = new Node(value); //create new node

			if (head == NULL)
			{
				head = newNode;                                  //if the
list is empty make newNode the head
				newNode->next = NULL;                   //next node of
head is null
			}
			else
			{
				//if the list is not empty make newNode at the end of
head
				Node* current = head;
				while (current->next != NULL)
				{
					current = current->next;
				}
				current->next = newNode;        //newNode is added at
the end of list
				newNode->next = NULL;                   //next will be
Null unless newNode is added


			}
		}
	}

	void groupOddEven()
	{
		//list is empty or contains only one node
		if (head == NULL || head->next == NULL)
			return;
```

```
            Node* evenHead = head->next;     //the next element of head is even
so it becomes evenHead
            Node* oddHead = head;                  //the head points to 1st
element which is odd so it becomes oddHead
            Node* evenCurrent = evenHead;
            Node* oddCurrent = oddHead;


            while (evenCurrent != NULL)
            {
                  //add the next of even node (which will be odd) to the next of
Odd
                  oddCurrent->next = evenCurrent->next;
                  oddCurrent = oddCurrent->next;

                  //add the next of odd node (which will be even) to the next of
even
                  evenCurrent->next = oddCurrent->next;
                  evenCurrent = evenCurrent->next;
            }

            //connect end of odd list with head of even list
            oddCurrent->next = evenHead;

      }

      //display
      void display()
      {
            Node* current = head;
            int i = 1;
            while (current != NULL)
            {
                  cout << "Value at node " << i++ << " is " << current->data <<
endl;
                  current = current->next;
            }
      }
};

int main()
{
      int num;
      List l;

      cout << "Enter number of nodes to create: ";
      cin >> num;
      l.create(num);
      cout << endl;
      l.display();

      l.groupOddEven();
      cout << endl << endl;
      cout << "AFTER GROUPING EVEN AND ODD:\n";
      l.display();

      system("pause");
      return 0;
}
```

```
Enter number of nodes to create: 5
Enter value for node 1 : 1
Enter value for node 2 : 2
Enter value for node 3 : 3
Enter value for node 4 : 4
Enter value for node 5 : 5

Value at node 1 is 1
Value at node 2 is 2
Value at node 3 is 3
Value at node 4 is 4
Value at node 5 is 5


AFTER GROUPING EVEN AND ODD:
Value at node 1 is 1
Value at node 2 is 3
Value at node 3 is 5
Value at node 4 is 2
Value at node 5 is 4
Press any key to continue . . .
```

Figure 4 Group ODD EVEN NODES

TASK 4

```
//Search any node in Linked List
#include <iostream>
using namespace std;
class Node
{
public:
	int data;
	Node* next;
	Node(int value) :data(value), next(NULL) {}
};

class List
{
public:
	Node* head;
	List() :head(NULL) {}
	void create(int numOfNodes)
	{
		for (int i = 0; i < numOfNodes; i++)
		{
			int value;
			cout << "Enter value for node " << i + 1 << " :";
			cin >> value;

			Node* newNode = new Node(value); //create new node

			if (head == NULL)
			{
				head = newNode;                                  //if the
list is empty make newNode the head
				newNode->next = NULL;                   //next node of
head is null
			}
			else
			{
				//if the list is not empty make newNode at the end of
head
				Node* current = head;
				while (current->next != NULL)
				{
					current = current->next;
				}
				current->next = newNode;        //newNode is added at
the end of list
				newNode->next = NULL;                   //next will be
Null unless newNode is added


			}
		}
	}

	void search(int value)
	{
		int findAll;
		cout << "Find all value (0) or First Found (1): ";
		cin >> findAll;

		Node* current = head;
		bool found = false;
```

```
		if (current == NULL)
		{
			cout << "Value not Found";
			return;
		}
		else
		{
			int i = 1;
			while (current != NULL)
			{
				if (current->data == value)
				{
					cout << "The Value " << value << " is found at
node " << i << endl;

					found = true;
					if (findAll == 1)		//if user has selected
to find only first value than it returns
						return;
				}
				current = current->next;
				i++;
			}
		}

		if (!found)
			cout << "Value not Found";
	}

	void display()
	{
		Node* current = head;
		int i = 1;
		while (current != NULL)
		{
			cout << "Value at node " << i++ << " is " << current->data <<
endl;
			current = current->next;
		}
	}
};

int main()
{
	int num;
	List l;

	cout << "Enter number of nodes to create: ";
	cin >> num;
	l.create(num);
	cout << endl;
	l.display();

	cout << "\nEnter value to find: ";
	cin >> num;

	cout << endl;
	l.search(num);

	cout << endl;
	system("pause");
	return 0;}
```

```
Enter number of nodes to create:
5
Enter value for node 1 :8
Enter value for node 2 :5
Enter value for node 3 :6
Enter value for node 4 :7
Enter value for node 5 :5

Value at node 1 is 8
Value at node 2 is 5
Value at node 3 is 6
Value at node 4 is 7
Value at node 5 is 5

Enter value to find: 6

Find all value (0) or First Found (1): 1
The Value 6 is found at node 3

Press any key to continue . . .
```

Figure 5 When first found value is selected.

```
Enter number of nodes to create: 6
Enter value for node 1 :5
Enter value for node 2 :8
Enter value for node 3 :9
Enter value for node 4 :8
Enter value for node 5 :1
Enter value for node 6 :8

Value at node 1 is 5
Value at node 2 is 8
Value at node 3 is 9
Value at node 4 is 8
Value at node 5 is 1
Value at node 6 is 8

Enter value to find: 8

Find all value (0) or First Found (1): 0
The Value 8 is found at node 2
The Value 8 is found at node 4
The Value 8 is found at node 6
```

Figure 6 When Find All Values is selected.

```
Enter number of nodes to create: 5
Enter value for node 1 :6
Enter value for node 2 :7
Enter value for node 3 :8
Enter value for node 4 :4
Enter value for node 5 :5

Value at node 1 is 6
Value at node 2 is 7
Value at node 3 is 8
Value at node 4 is 4
Value at node 5 is 5

Enter value to find: 200

Find all value (0) or First Found (1): 1
Value not Found
Press any key to continue . . .
```

Figure 7 When value is not Found in the List

TASK 5

```
//Program to check palindrome in a linked list
#include <iostream>
using namespace std;
//Node Class
class Node
{
public:
	int data;
	Node* next;
	Node(int value) :data(value), next(NULL) {}
};
class List
{
public:
	Node* head;
	List() :head(NULL) {}
	void create(int numOfNodes)
	{
		for (int i = 0; i < numOfNodes; i++)
		{
			int value;
			cout << "Enter value for node " << i + 1<<" :";
			cin >> value;

			Node* newNode = new Node (value);	//create new node

			if (head == NULL)
			{
				head = newNode;					//if the
list is empty make newNode the head
				newNode->next = NULL;			//next node of
head is null
			}
			else
			{
				//if the list is not empty make newNode at the end of
head
				Node* current = head;
				while (current->next != NULL)
				{
					current = current->next;
				}
				current->next = newNode;	//newNode is added at
the end of list
				newNode->next = NULL;			//next will be
Null unless newNode is added


			}
		}
	}

	bool isPalindrome()
	{
		Node* copyHead = head;
		Node* copyCurrent = copyHead;
		Node* current = head;

		//creating a copy of the linked list
		while (current != NULL)
```

```
		{
			copyCurrent = current->next;
			current = current->next;
		}

		//reversing the copyCurrent list
		copyCurrent = copyHead;				//assigning Head again because it came on the last node while making copy
		Node* prev = NULL, * nextNode = NULL;
		while (copyCurrent != NULL)
		{
			nextNode = copyCurrent->next;	//the next of the current node is made nextNode
			copyCurrent->next = prev;		//current nodes now points to previous node
			prev = copyCurrent;				//current is made prevvious so that when we move forward we have the address of previous
			copyCurrent = nextNode;			//nextNode is made current to move forward
		}
		copyHead = prev;					//prev points to newHead of reverse loop


		//checks for palindrome by comparing the nodes of both reversed and original list.
		copyCurrent = copyHead;
		current = head;

		bool palindrome = false;
		while (current != NULL)
		{
			if (copyCurrent->data == current->data)
			{
				copyCurrent = copyCurrent->next;
				current = current->next;
				palindrome = true;
			}
			else
				return false;
		}

		return palindrome;
	}

	//display
	void display()
	{
		Node* current = head;
		int i = 1;
		while (current != NULL)
		{
			cout << "Value at node " << i++ << " is " << current->data << endl;
			current = current->next;
		}
	}
};

int main()
{
	int num;
	List l;
```

```
	cout << "Enter number of nodes to create: ";
	cin >> num;
	l.create(num);
	cout << endl;
	l.display();

	cout << endl << endl;
	if (l.isPalindrome())
		cout << "The List is Palindrome\n";
	else
		cout << "The List is not Palindrome\n";


	system("pause");
	return 0;

}
```

```
Enter number of nodes to create: 5
Enter value for node 1 :1
Enter value for node 2 :2
Enter value for node 3 :3
Enter value for node 4 :2
Enter value for node 5 :1

Value at node 1 is 1
Value at node 2 is 2
Value at node 3 is 3
Value at node 4 is 2
Value at node 5 is 1


The List is Palindrome
Press any key to continue . . .
```

Figure 9 Palindrome

```
Enter number of nodes to create: 5
Enter value for node 1 :1
Enter value for node 2 :5
Enter value for node 3 :8
Enter value for node 4 :6
Enter value for node 5 :4

Value at node 1 is 1
Value at node 2 is 5
Value at node 3 is 8
Value at node 4 is 6
Value at node 5 is 4


The List is not Palindrome
Press any key to continue . . .
```

Figure 8 Not Palindrome

TASK 6

```
//Program to Add/Remove song from a playlist
#include <iostream>
#include<string>
using namespace std;
//Node Class of playlist
class Node
{
public:
    string songName;
    string artist;
    Node* next;

    Node(string song, string artist) : next(nullptr), songName (song),
artist(artist){}
};
class List
{
public:
    Node* head;
    List() : head(nullptr) {}

    //print
    void print()
    {
        Node* current = head;

        //if list is empty
        if (current == NULL)
        {
            cout << "Playlist is empty." << endl;
            return;
        }


        int i = 0;
        while (current != NULL)
        {
            cout << "Song " << i++ << ": " << current->songName << " by " <<
current->artist << endl;
            current = current->next;
        }
    }

    void addSong(int position)
    {
        //if position is wrong
        if (position < 0)
        {
            cout << "Invalid position" << endl;
            return;
        }

        string song, artist;
        cout << "Enter song name: ";
        cin >> song;
        cout << "Enter artist: ";
        cin >> artist;

        Node* newNode = new Node(song,artist);
```

```
    //if position is zero song is added at first place (zero place)
    if (position == 0)
    {
        newNode->next = head;
        head = newNode;
    }
    else
    {
        Node* current = head;
        int currentPosition = 0;

        while (current && currentPosition < position - 1)
        {
            current = current->next;
            currentPosition++;
        }

        if (current == NULL)
        {
            cout << "Invalid position. Song not added." << endl;
            return;
        }

        //moves to next
        newNode->next = current->next;
        current->next = newNode;
    }

    cout << "Song added successfully!" << endl;
}

void removeSong(int position)
{
    if (position < 0)
    {
        cout << "Invalid position" << endl;
        return;
    }

    if (head == NULL)
    {
        cout << "Playlist is empty" << endl;
        return;
    }

    if (position == 0)
    {
        Node* temp = head;
        head = head->next;
        cout << "Song removed successfully!" << endl;
        return;
    }

    Node* current = head;
    int currentPosition = 0;

    //reching pos - 1
    while (current && currentPosition < position - 1)
    {
        current = current->next;
        currentPosition++;
    }
```

```
        //if pos-1 is pointing to NULL means there is no song ahead
        if (current == nullptr || current->next == nullptr)
        {
            cout << "Invalid position. Song not removed." << endl;
            return;
        }

        Node* temp = current->next;
        current->next = temp->next;

        cout << "Song removed successfully!" << endl;
    }

};

int main()
{

    List l;
    int pos;
    int option =0;
    while (option != 4)
    {
        cout << "1. Add Song\n2. Remove Song\n3. View Song List\n4. Exit\nInput:
";
        cin >> option;

        switch (option)
        {
            //Add Song
        case 1:
            do
            {
                cout << "Enter Song Position: or -1 to Return: ";
                cin >> pos;

                if (pos != -1)
                    l.addSong(pos);

            } while (pos != -1);
            break;

            //Remove Song
        case 2:
            do
            {
                cout << "Enter Song Position to Delete: or -1 to Return: ";
                cin >> pos;

                if (pos != -1)
                    l.removeSong(pos);

            } while (pos != -1);
            break;
        case 3:
            cout << "\nThe Song list is : \n";
            l.print();
            break;
        case 4:
            return 0;
        }

        cout << endl;
```

```
    }

    return 0;
}
```

```
1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 1
Enter Song Position: or -1 to Return: 0
Enter song name: Baarish
Enter artist: Atif
Song added successfully!
Enter Song Position: or -1 to Return: 1
Enter song name: Sassi
Enter artist: Asim
Song added successfully!
Enter Song Position: or -1 to Return: 2
Enter song name: Malang
Enter artist: Aima
Song added successfully!
Enter Song Position: or -1 to Return: 4
Enter song name: song
Enter artist: song
Invalid position. Song not added.
Enter Song Position: or -1 to Return: -1

1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 3

The Song list is :
Song 0: Baarish by Atif
Song 1: Sassi by Asim
Song 2: Malang by Aima

1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 2
Enter Song Position to Delete: or -1 to Return: 5
Invalid position. Song not removed.
Enter Song Position to Delete: or -1 to Return: 2
Song removed successfully!
Enter Song Position to Delete: or -1 to Return: -1
```

```
1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 3

The Song list is :
Song 0: Baarish by Atif
Song 1: Sassi by Asim

1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 1
Enter Song Position: or -1 to Return: 2
Enter song name: Excuses
Enter artist: AP_Dhillion
Song added successfully!
Enter Song Position: or -1 to Return: -1
```

```
1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 3

The Song list is :
Song 0: Baarish by Atif
Song 1: Sassi by Asim
Song 2: Excuses by AP_Dhillion

1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 1
Enter Song Position: or -1 to Return: 1
Enter song name: Iraaday
Enter artist: Hannan
Song added successfully!
Enter Song Position: or -1 to Return: -1
```

```
1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 3

The Song list is :
Song 0: Baarish by Atif
Song 1: Iraaday by Hannan
Song 2: Sassi by Asim
Song 3: Excuses by AP_Dhillion

1. Add Song
2. Remove Song
3. View Song List
4. Exit
Input: 4
```